

Prepared by: Kirolos Mina

Supervised by: Dr. Rami Zewail

CSE 521: Embedded ML

Introduction to JAX

Content	Page
Overview	1
Installation	2
JAX vs. Numpy	2
JAX common functions	4
Examples	6

➤ **Overview: JAX is a library for array-oriented numerical computation with automatic differentiation and JIT compilation to enable high-performance machine learning research.**

- JAX provides a unified Numpy-like interface to computations that run on CPU, GPU or TPU in local or distributed settings.
- JAX features built-in Just-In-Time compilation via Open XLA, an open-source machine learning compiler ecosystem.
- JAX functions support efficient evaluation of gradients via its automatic differentiation transformations.
- JAX functions can be automatically vectorized to efficiently map them over arrays representing batches of inputs

➤ **Installation:** JAX can be installed for CPU on linux, windows and macOS directly from the python package index.

Using JAX requires installing two packages: **jax**, which is pure Python and cross-platform, and **jaxlib** which contains compiled binaries, and requires different builds for different operating systems and accelerators.

- **For CPU-only**

pip install jax

- **For or Nvidia GPU**

pip install -U "jax[cuda13]"

- **For GPU (AMD, ROCm)**

pip install -U "jax[rocm7-local]"

- **For TPU (Google cloud TPU VM)**

pip install -U "jax[tpu]"

➤ **JAX vs. NumPy :** JAX provides a NumPy-inspired interface for convenience, and unlike NumPy arrays, JAX arrays are always immutable.

- JAX provides `jax.numpy` which closely mirrors the NumPy API and provides easy entry into JAX, almost anything that can be done with numpy can be done with `jax.numpy` which is typically imported under the `jnp` alias

import jax.numpy as jnp

- Python's duck-typing allows JAX arrays and NumPy arrays to be used interchangeably in many places. However, there is one important difference between JAX and NumPy arrays: JAX arrays are immutable, meaning that once created their contents cannot be changed.

```
# NumPy: mutable arrays
x = np.arange(10)
x[0] = 10
print(x)
```

```
[10  1  2  3  4  5  6  7  8  9]
```

```
# JAX: immutable arrays
x = jnp.arange(10)
x[0] = 10
```

`TypeError: JAX arrays are immutable and do not support in-place item assignment.`

For updating individual elements, JAX provides an indexed update syntax that returns an updated copy:

```
y = x.at[0].set(10)
print(x)
print(y)
```

```
[0 1 2 3 4 5 6 7 8 9]
[10 1 2 3 4 5 6 7 8 9]
```

- Out-of-bounds indexing:

```
np.arange(10)[11]
```

`IndexError: index 11 is out of bounds for axis 0 with size 10`

```
jnp.arange(10)[11]
```

```
Array(9, dtype=int32)
```

➤ JAX common functions

1- Jax.array function

- Create arrays using JAX API functions
- JAX array objects have a `devices` attribute that indicate where the array is stored
- JAX arrays can be sharded across multiple devices for parallel computation

We typically don't call the `jax.array` constructor directly, but rather create arrays via JAX API functions. For example, `jax.numpy` provides familiar NumPy-style array construction functionality such as `jax.numpy.zeros`, `jax.numpy.linspace`, `jax.numpy.arange`

```
import jax
import jax.numpy as jnp

x = jnp.arange(5)
isinstance(x, jax.Array)
```

```
True
```

- JAX array objects have a `devices` method that lets you inspect where the contents of the array are stored. In general, an array may be sharded across multiple devices, in a manner that can be inspected via the `sharding` attribute

```
x.devices()
```

```
{CpuDevice(id=0)}
```

```
x.sharding
```

```
SingleDeviceSharding(device=CpuDevice(id=0), memory_kind=device)
```

2- Jax.jit function

- By default, JAX executes operations one at a time, in sequence
- Using a just-in-time (JIT) compilation decorator, sequences of operations can be optimized together and run at once.
- Not all JAX code can be JIT compiled, as it requires array shapes to be static and known at compile time

```
import jax.numpy as jnp

def norm(X):
    X = X - X.mean(0)
    return X / X.std(0)
```

```
from jax import jit
norm_compiled = jit(norm)
```

3-jax.grad function

- JAX provides automatic differentiation via the jax.grad transformation
- Jax.grad and jax.jit transformations compose and can be mixed arbitrarily

```
from jax import grad

def sum_logistic(x):
    return jnp.sum(1.0 / (1.0 + jnp.exp(-x)))

x_small = jnp.arange(3.)
derivative_fn = grad(sum_logistic)
print(derivative_fn(x_small))
```

```
[0.25      0.19661197 0.10499357]
```

Some another useful functions can be used such as vmap for vectorization, random for pseudo numbers and debug for debugging.

➤ Examples [control flow and logical operators with JIT]

- When executing eagerly (outside of jit), JAX code works with python control flow and logical operators just like NumPy code.
- Using control flow and logical operators with jit is more complicated.
- In a nutshell, python control flow and logical operators are evaluated at JIT compile time, such that the compiled function represents a single path through the control flow graph.

- If the path depends on the values of the inputs, the function by default cannot be JIT compiled. The path may depend on the shape or data type of inputs, and the function is re-compiled every time it is called on an input with a new shape or data type.

```
@jit
def f(x):
    for i in range(3):
        x = 2 * x
    return x

print(f(3))
```

24

```
@jit
def g(x):
    y = 0.
    for i in range(x.shape[0]):
        y = y + x[i]
    return y

print(g(jnp.array([1., 2., 3.])))
```

6.0

```

@jit
def f(x):
    if x < 3:
        return 3. * x ** 2
    else:
        return -4 * x

# This will fail!
f(2)

```

```

@jit
def g(x):
    return (x > 0) and (x < 3)

# This will fail!
g(2)

```

Structured control flow primitives: 1- lax.cond

python equivalent:

```

def cond(pred, true_fun, false_fun, operand):
    if pred:
        return true_fun(operand)
    else:
        return false_fun(operand)

```

```

from jax import lax

operand = jnp.array([0.])
lax.cond(True, lambda x: x+1, lambda x: x-1, operand)
# --> array([1.], dtype=float32)
lax.cond(False, lambda x: x+1, lambda x: x-1, operand)
# --> array([-1.], dtype=float32)

```

```

Array([-1.], dtype=float32)

```


2- While_loop

python equivalent:

```
def while_loop(cond_fun, body_fun, init_val):  
    val = init_val  
    while cond_fun(val):  
        val = body_fun(val)  
    return val
```

```
init_val = 0  
cond_fun = lambda x: x < 10  
body_fun = lambda x: x+1  
lax.while_loop(cond_fun, body_fun, init_val)  
# --> array(10, dtype=int32)
```

```
Array(10, dtype=int32, weak_type=True)
```

3- Fori_loop

python equivalent:

```
def fori_loop(start, stop, body_fun, init_val):  
    val = init_val  
    for i in range(start, stop):  
        val = body_fun(i, val)  
    return val
```

```
init_val = 0  
start = 0  
stop = 10  
body_fun = lambda i,x: x+i  
lax.fori_loop(start, stop, body_fun, init_val) .  
# --> array(45, dtype=int32)
```

```
Array(45, dtype=int32, weak_type=True)
```

Logical operators

```
def python_check_positive_even(x):
    is_even = x % 2 == 0
    # `and` short-circuits, so when `is_even` is `False`, `x > 0` is not evaluated.
    return is_even and (x > 0)

@jit
def jax_check_positive_even(x):
    is_even = x % 2 == 0
    # `logical_and` does not short circuit, so `x > 0` is always evaluated.
    return jnp.logical_and(is_even, x > 0)

print(python_check_positive_even(24))
print(jax_check_positive_even(24))
```

```
True
True
```

```
x = jnp.array([-1, 2, 5])
print(jax_check_positive_even(x))
```

```
[False  True False]
```

If you just want to apply grad to your python functions, without jit, you can use regular Python control-flow constructs with no problems.

```
def f(x):
    if x < 3:
        return 3. * x ** 2
    else:
        return -4 * x

print(grad(f)(2.)) # ok!
print(grad(f)(4.)) # ok!
```

```
12.0
-4.0
```